Numerical Contracts for Routing Solution Checkers

Contract compatibility, arithmetic alternatives and the design of a computational standard

Florian Rascoussier*

Research report, revised 14 October 2026

Abstract

A routing solution checker defines the finite computation used to evaluate feasibility, compare incumbents and report optimization objectives. Changing its arithmetic can therefore change a benchmark contract even when the mathematical formulation is unchanged. We study the breaking transition from MAMUT-routing-lib 0.11.0 to 0.12.0, which replaced parts of the time-dependent composition protocol and introduced exact-decimal ranking for static incumbents. We reproduce both released computations, compare fixed routes with an independent rational reference, and investigate the available benchmark store. Both contracts accept the 898 original routes, but 31 of their 146 solution totals differ. The newer computation improves several fidelity measurements without providing a general exact-arithmetic contract. Static ranking differences are small in value but can change which incumbent is retained. We recommend restoring the historical time-dependent protocol for v0.13, preserving its limitations and the independent integrity repairs. We also examine numerical evaluation as a multiobjective design problem: binary64, integer scaling, rational arithmetic and higher or adaptive precision offer different balances of simplicity, expressiveness, computational cost and fidelity to declared data. Controlled Python measurements inform these trade-offs without claiming production solver performance.

Keywords: Vehicle routing, numerical contracts, time dependence, floating-point arithmetic, exact arithmetic, benchmark reproducibility.

Contents

1 Introduction **3**

2 What the numerical contracts define **3**

3 Comparison method and principal evidence **4**

4 Arithmetic choice as a design problem **6**

5 Recommendation and limits **7**

References **8**

*IMT Atlantique, Lab-STICC, CNRS, UMR 6285 (équipe DECIDE) and INSA Lyon, Inria, CITI, UR3720, 69621 Villeurbanne, France (équipe EMERAUDE). ORCID identifier: 0009-0005-3253-9814.

A The 2 time-dependent recipes **11**

A.1 Generic composition and interpolation 11

A.2 Release 0.11.0: generic sequential folding 11

A.3 Release 0.12.0: restriction, direct ready times and a slope shortcut 11

A.4 A qualified exactness claim 14

B Independent exact models and measured alternatives **14**

B.1 Forward partitioning instead of composition 14

B.2 Isolated changes and a stricter slope predicate 15

B.3 Alternatives analysed but not installed 16

C Experimental protocol and coverage **16**

D Original Dabia2013 solutions **17**

E Stepwise examples and the broader TD store **18**

F The static checker and incumbent ranking **20**

G Consequences of the September change campaign **22**

H Literature and historical choices **23**

I Original solution totals **24**

J Finite-input models and portability **26**

J.1 The finite input model 26

J.2 Conformance, fidelity and reproducibility 26

J.3 The role of a reference checker 27

K Provenance and reproduction **27**

L Historical static candidates and controlled measurements **28**

1 Introduction

This report examines how a numerical checker becomes part of an optimization benchmark's definition, and what follows when that checker changes. The MAMUT-routing ecosystem provides routing instances, Best-Known Solutions (BKS), validation tools and a website for their distribution and comparison. Its shared library, MAMUT-routing-lib, supplies the cost and feasibility computations used by these components. Hence its checker is more than a final consistency test: together with the objective and incumbent-ranking policy, it determines what the ecosystem reports as a valid solution or an improvement [ANR26a; ANR26b; ANR26c].

We consider the Capacitated Vehicle Routing Problem (CVRP), the Vehicle Routing Problem with Time Windows (VRPTW), and their Time-Dependent (TD) counterparts, particularly the Time-Dependent Vehicle Routing Problem with Time Windows (TDVRPTW). In the time-dependent setting, the objective optimizes each route's depot departure to minimize its duration, then aggregates route durations under a prescribed rule. Solvers may use faster internal evaluation structures, but the reference checker defines the authoritative reported value. A change to that authority can affect accepted moves, stored incumbents, reference gaps and the interpretation of historical bounds or proofs [Ras26a; Ras26b; Ras26c].

The transition to library version 0.12 introduced a breaking numerical change. The preceding protocol, represented here by frozen release 0.11, applies one generic composition operation throughout the time-dependent route fold. Version 0.12 replaces several compositions with specialized transformations and introduces a unit-slope interpolation shortcut. It also changes static BKS ranking to an exact sum of decimal interpretations, although the ordinary static checker remains unchanged. The September campaign repriced incumbents and revised some route sets under these choices. We therefore need to distinguish a more faithful approximation to a chosen mathematical model from a compatible implementation of the established benchmark contract [ANR26b; ANR26c].

Our first contribution is a reproducible comparison of these contracts, including their numerical limitations and consequences for the benchmark store. Our second is a design analysis of the alternatives: the choice of arithmetic must balance fidelity with computational cost, representational freedom, reproducibility and implementation complexity. We use this evidence to recommend a scoped restoration of the historical TD protocol in v0.13. Static ranking is evaluated separately, since its effects and implementation costs differ from those of TD composition. All experiments evaluate fixed routes or previously available candidates, without new solver searches or renewed optimality proofs.

2 What the numerical contracts define

This section separates the input model, the evaluation recipe and the ranking policy. A complete contract maps byte-defined instance artifacts and a solution to a feasibility verdict and an objective value. It specifies parsing, arithmetic operations, comparisons, branch conventions and reduction order. The exact mathematical objective over ideal data, an exact objective over stored operands and the checker's computed value are distinct objects. A BKS is an incumbent under one such contract, not an optimality certificate by itself.

A stored binary64 value is an exact dyadic rational. We can therefore evaluate the exact piecewise-affine model through those stored operands without introducing floating-point roundoff. Alternatively, we can treat their shortest decimal spellings as exact. These interpretations differ, just as the binary64 value written as `0.1` differs from the rational $1/10$. Neither interpretation reconstructs an unrounded road model or an ideal geometric distance. Our exact TD references use materialized Arrival-Time Functions (ATFs), so their exactness concerns those declared finite inputs and domain conventions [Pyt26a].

For TD routes, both releases fold arc ATFs and customer ready-time maps sequentially. The historical `td-fold/1` represents waiting and service through the same composition primitive as the arcs. The newer `td-fold/2` directly restricts the first arc and transforms ready times, with a slope shortcut based on equality of rounded endpoint differences. These replacements change the operation sequence and may change later breakpoint positions, branch choices and departures. Appendix A gives both algorithms and their numerical primitives. Their shared use of binary64 does not make them the same contract.

A simple witness illustrates the trade-off. With arrival 1009, service 5 and window $[250, 12360]$, the generic interpolation of the ready-time map returns 1014.0000000000001, whereas direct addition returns 1014. The latter agrees with the intended waiting/service map in this example. However, changing a computed value by one representable step can move a later composition across an ATF jump. Conversely, the newer rounded unit-slope test can classify a mathematically non-unit segment as unit and worsen agreement with exact interpolation. Specialized arithmetic improves some cases without removing the need for a declared contract and boundary policy.

For static routing, the released checker traverses submitted routes and adds every arc into one sequential accumulator. Version 0.12 preserves that computation, but stores canonically ordered routes and ranks candidates by exact sums of their decimal-interpreted arc costs. Thus the displayed checker cost and ranked objective can differ. We compare the historical submitted-order policy, canonical binary64 scoring and exact-decimal ranking separately, retaining the fleet tier of hierarchical objectives. Accurate summation alone is another possible contract, not an implementation of the prescribed sequential sum [Pyt26b; ORO05].

3 Comparison method and principal evidence

This section reports the decisive fixed-route evidence and the controlled implementation comparisons. We freeze the released sources, instance artifacts and candidate routes. An independent rational oracle partitions the departure domain by forwarding boundaries through affine pieces, rather than importing the checker composition routine. It distinguishes breakpoint points from open intervals and attained minima from unattained infima. Appendix B documents the oracle and isolated changes. Appendix C gives coverage, selection and conformance checks. Source and distribution hashes, including the wheel-to-tag verification, appear in Appendix K.

The original panel contains 146 published Lera-Romero solutions and 898 routes on the canonical Dabia2013 artifacts. Both releases accept every route and solution. Their totals differ for 31 solutions, while each exactly reproduces 48 published totals. Matching historical published totals therefore does not identify a numerical winner. Against the correctly rounded exact optimum over binary operands, the historical fold matches 455 route values and the newer fold 494, with maximum absolute discrepancy about 7.79×10^{-12} for both. We preserve these improvements when assessing the restoration recommendation [LMS20; Dab+13; Ras26d].

A composed route value and a returned executable departure also require separate checks. Generic forward replay rejects 30 historical-fold departures and 25 newer-fold departures on the original panel. Including feasible schedules whose replayed duration differs from the composed value gives 154 and 77 discrepancies. These counts concern specific rounded departure witnesses, not whole-route infeasibility. An exact optimum match likewise does not establish that a representable departure attains that optimum. The detailed diagnostics remain in Appendix D.

The available frozen store contains 1,956 static and 3,052 TD records. There are 20 missing static and 20 missing TD sidecars, retained in coverage denominators. On the available TD records, 871 totals change between the contracts, with maximum absolute difference 25 and no whole-route temporal-feasibility flips. The Rifki-30 witness changes from duration/departure $(1768, 1301)$ to $(1743, 1341)$, with the newer result agreeing with the rational reference. The 35-record broader

exact panel contains 243 routes, and both exact input interpretations complete. These targeted and stored-route results establish incompatibility and several fidelity gains, without certifying every boundary or a new optimum.

Static summation is sensitive to route order on 1,069 records in the tested permutations. The grouped cost helper differs from the flat checker on 924 records. Nevertheless, the maximum observed difference between a submitted flat total and its exact-decimal sum is about 2.57×10^{-7}, with maximum relative difference about 5.01×10^{-15}. All 818 tested integer-cost records agree exactly. Among the 31 campaign restorations that tie in exact-decimal cost, canonical binary64 scoring distinguishes 15 pairs, with maximum gap about 3.49×10^{-10}. Small value differences can therefore have a discrete incumbent-selection consequence. Appendix F retains the full sweep.

The additional accessible static history provides 2,273 distinct ordered candidates on 1,956 instances, with 317 candidate pairs. Both policies agree on all 286 strict decimal improvements. The remaining 31 pairs are decimal ties distinguished by historical submitted-order scoring, with maximum gap $8.15e-10$. Canonical binary64 scoring distinguishes 15 of these pairs. There are no strict ranking reversals in this accessible history. We exclude the unavailable third Mamut2026 campaign rather than infer its outcome.

We measure preloaded route folds and complete released checkers separately, using 7 timing rounds with alternating release order on one Linux x86-64 host under CPython 3.13.13. Setup is outside the checker timing and is recorded separately. Table 1 reports panel medians and paired time ratios. The newer release is faster in these measurements, so efficiency does not support calling the historical TD implementation faster. Its restoration is instead motivated by contract continuity and the simpler shared composition protocol.

Table 1: Controlled Python measurements. Times are medians in milliseconds, first within each record's repeated measurements and then across records. Ratios are medians of paired 0.12/0.11 times, rather than ratios of the displayed panel medians. Setup, file loading and process startup are excluded.

Panel and evaluation	Records	0.11 (ms)	0.12 (ms)	Paired ratio
Original route folds	898	0.103	0.068	0.668
Original full checkers	146	0.554	0.414	0.733
Broader route folds	243	0.823	0.607	0.698
Broader full checkers	35	8.270	5.731	0.705

The static timing panel contains 25 records selected at fixed size positions within each family. Forming and comparing two exact-decimal objectives takes a median 152.5 times the corresponding sequential binary64 reduction and comparison. This includes the released decimal-conversion primitive and rational additions, not the full store's validation or filesystem work. It is not a solver speed-up estimate. Appendix L gives the protocol, dispersion and per-record reduction timings. Accurate summation and integer scaling are measured separately, with their distinct input and output semantics.

The bounded rational timing panel selects 24 routes of at most 25 customers, 12 from each TD panel, by a fixed hash rule. Both input interpretations complete on 24 selected routes. On these matched routes, the median binary-rational-reference/historical-fold time ratio is 315.2. This comparison measures our independent partitioning reference, including stage construction, against the preloaded composition kernel. The difference includes algorithmic structure and arithmetic costs. It cannot isolate a universal rational-arithmetic penalty or predict an optimized exact solver.

4 Arithmetic choice as a design problem

This section considers the alternatives beyond the two released implementations. We treat contract design as a multiobjective problem because reducing one cost can increase another: fewer numerical branches simplify implementation and solver parity, exact arithmetic increases fidelity to declared operands, and a fixed integer scale can improve arithmetic predictability while restricting resolution. Expressiveness also matters, including fractional costs, non-unit slopes, discontinuities and exact equality at deadlines. The measured implementations occupy different points in this design space. We do not claim to compute a Pareto frontier over all possible designs.

Binary64 retains a broad dynamic range and direct hardware support, but reproducibility requires a prescribed operation sequence and comparison policy. More accurate reduction, including compensated summation or `fsum`, can reduce static accumulation error without changing the stored arc operands. It does not repair roundoff inside TD composition or determine how to rank exact ties. These methods remain approximate at their finite output precision and must be identified if they replace historical scoring [Gol91; ORO05; Pyt26b].

Integer scaling makes addition exact when every operand lies on the chosen grid and every finite-width intermediate fits its range. If an input does not lie on that grid, rounding it introduces quantization and a fixed resolution. Increasing the scale reduces that loss but consumes representable range. Arbitrary-size integers avoid fixed-width overflow, at a computational cost. General TD interpolation and boundary preimages can remain nonintegral even when input breakpoints are integers, so integer addition alone does not supply an exact general TD model. In our static panel, scale 1000 exactly preserves every consumed decimal arc and the objective within signed-64-bit range on 1,488 records. This is an eligibility result for consumed operands, not a range proof for all solver penalties or intermediates [Ras+26].

PyVRP provides a practical illustration of this trade-off. Issue #623 discusses awkward fractional-cost modeling, integer overflow and sentinel handling after earlier integer choices. The project subsequently merged PR #1196, returning to floats, on September 30, 2026. This history shows that resolution, range, modeling convenience and numerical comparisons influence a solver's arithmetic design. It does not establish that either arithmetic type is universally preferable, nor that PyVRP's measurements transfer to this checker [Wou24; Wou26].

Rational arithmetic supplies exact comparisons and interpolation for a specified finite rational input model, with arbitrary-size numerator and denominator growth. It still requires a domain convention, a departure representation and a policy for rounding reported values. Exact binary and decimal references differ after output rounding on 377 original routes, despite agreeing on whole-route feasibility. Exactness therefore requires choosing what is exact. Our measured independent oracle offers calibration and counterexample analysis, while its more expensive partitioning algorithm is not a prediction of rational production-solver throughput [ZS22; ZNS23].

Higher binary or decimal precision can reduce many errors while retaining finite rounding. Libraries such as MPFR provide controlled precision and rounding semantics, but correctly rounding each operation does not make a complete chain of finite operations exact. Interval or ball arithmetic can enclose values and certify separated comparisons. Adaptive precision with exact fallback is another option, especially when most comparisons are easy and a minority lie near boundaries. Exact ties and uncertain jump membership need explicit escalation or an unresolved outcome. Finally, storing segment intent explicitly can avoid inferring a unit slope from rounded endpoints, at the cost of changing the artifact representation. These options are conceptual in this report. Appendix B discusses their obligations [MPF25; Joh17].

5 Recommendation and limits

This section connects the measured trade-offs to a scoped v0.13 recommendation. We recommend restoring the frozen historical `td-fold/1` protocol as the authoritative TD cost calculation, preserving the earlier releases and retaining independently justified integrity and interface repairs. The rationale is continuity of the benchmark objective and a simpler implementation based on one reusable composition primitive. It is not a claim that the old computation is mathematically exact or that the newer implementation provides no benefit. Its known jump amplification and departure-witness limitations remain part of the report and conformance fixtures.

The restoration should specify the fold, input interpretation, comparisons, canonical TD aggregation and departure semantics. Incumbents must be repriced and selected under that declared contract while retaining their original values and source identities. Proof metadata must remain attached to the computation used by the prover. Repricing an incumbent does not transfer a lower bound or renew an optimality proof. Cache entries, saved runs, website history and reference scores likewise need the evaluator identity as well as the instance hash. Appendix G separates these consequences from independent repairs.

For static v0.13, we recommend ranking incumbents by the authoritative checker value while retaining canonical route ordering for storage. Candidates and incumbents should be validated and repriced in that same stored order before comparing their objective keys, preserving the fleet tier where applicable. This removes the separate exact-decimal ranking objective while keeping deterministic storage and candidate-validation repairs. It restores checker-aligned ranking, not every detail of the 0.11 storage policy. Historical declared values remain attached to their original serialized route order. Canonicalization does not make sequential addition order-independent, and route reversal or a different arc multiset can still distinguish exact-decimal ties. The rational sums remain available as diagnostics rather than a second authoritative BKS objective.

The controlled timings concern Python checkers, reduction kernels and the independent oracle on declared panels. We have not measured a migrated KAYROS solver, searched new routes or recertified historical proofs. Same-host interpreter parity and selected C++ expression diagnostics are not a cross-platform route-engine certificate. The missing sidecars and unavailable later campaign remain explicit gaps. These limits do not prevent a compatibility recommendation, but they bound efficiency, feasibility and optimality claims.

The wider conclusion is that a computational standard must state its choices and preserve the evidence needed to revisit them. Exact references can coexist with an efficient operational authority without silently replacing its objective. Changing arithmetic, input resolution or incumbent ranking is a design decision whose benefits and obligations should be evaluated together.

Acknowledgements

We thank Adrien Pichon for the collaboration on MAMUT-routing and the availability of the library implementations examined here. We thank Gonzalo Lera-Romero for making the original solver and solution records available, and the contributors to the benchmark collections and numerical software used in this investigation. This work is funded by the French National Research Agency (ANR) as part of the MAMUT project, ANR-22-CE22-0016, “Machine learning And Matheuristics algorithms for Urban Transportation”.

Data Availability

The experiment code, frozen checker sources, compact input panels, candidate operands, numerical results and report sources are publicly available at https://github.com/0nyr/routing-numerical-contracts. Original code is MIT licensed, while retained source and benchmark data preserve their upstream terms. Appendix K explains the archived evidence and external-input coverage. No public optimality certificate is created by this investigation.

Declaration of AI Use

Generative AI tools were used throughout the preparation of this work, including GPT 6.1 Sol (OpenAI). They contributed to the experiment tooling, to experiment orchestration and analysis, and to the drafting of this report. Florian Rascoussier reviewed and revised all outputs from these tools and takes full responsibility for their use in the final content of the work.

References

[ANR26a] ANR MAMUT. *MAMUT-routing: Benchmark Catalog, Solution Store and Publication Tools*. Frozen ecosystem source snapshot. 2026. URL: https://github.com/ANR-MAMUT/MAMUT-routing/tree/0d48b4f383c4336af7de8a2f2cac0e8502da6c83.

[ANR26b] ANR MAMUT. *MAMUT-routing-lib v0.11.0*. Frozen source commit 425dff3. 2026. URL: https://github.com/ANR-MAMUT/MAMUT-routing-lib/tree/425dff351fdd9522ea6bb8c537dd416dfcd19614.

[ANR26c] ANR MAMUT. *MAMUT-routing-lib v0.12.0*. Frozen source commit a9f7d9a. 2026. URL: https://github.com/ANR-MAMUT/MAMUT-routing-lib/tree/a9f7d9a718b1d7d2b5db6ccc470d6c20065ec836.

[Ras26a] Florian Rascoussier. *KAYROS: An Anytime and Exact Open-Source Solver for Duration-Minimization Time-Dependent Vehicle Routing. A Technical Report and a Case Study in Human-AI Engineering*. 2026. arXiv: 2607.23116 [math.OC]. URL: https://arxiv.org/abs/2607.23116.

[Ras26b] Florian Rascoussier. *KAYROS: An Anytime and Exact Solver for the Time-Dependent Vehicle Routing Problem with Time Windows*. Extended version. Local source revision dated 2026-09-29. 2026. arXiv: 2609.29597. URL: https://arxiv.org/abs/2609.29597.

[Ras26c] Florian Rascoussier. *Anytime Solver Evaluation with a Normalized Signed Primal Integral and Explicit Reference Policies*. Extended version. 2026. arXiv: 2608.18288. URL: https://arxiv.org/abs/2608.18288.

[Pyt26a] Python Software Foundation. *Rational numbers: fractions*. 2026. URL: https://docs.python.org/3/library/fractions.html (visited on 2026/09/30).

[Pyt26b] Python Software Foundation. *Mathematical functions: math.fsum*. 2026. URL: https://docs.python.org/3/library/math.html#math.fsum (visited on 2026/09/30).

[ORO05] Takeshi Ogita, Siegfried M. Rump, and Shin'ichi Oishi. "Accurate Sum and Dot Product". In: *SIAM Journal on Scientific Computing* 26.6 (2005), pp. 1955–1988. DOI: 10.1137/030601818.

[LMS20] Gonzalo Lera-Romero, Juan J. Miranda Bront, and Francisco J. Soulignac. "Linear edge costs and labeling algorithms: The case of the time-dependent vehicle routing problem with time windows". In: *Networks* 76.1 (2020/07), pp. 24–53. DOI: 10.1002/net.21937. URL: https://onlinelibrary.wiley.com/doi/10.1002/net.21937.

[Dab+13] Said Dabia, Stefan Ropke, Tom Van Woensel, and Ton De Kok. "Branch and Price for the Time-Dependent Vehicle Routing Problem with Time Windows". In: *Transportation Science* 47.3 (2013/08), pp. 380–396. DOI: 10.1287/trsc.1120.0445. URL: https://pubsonline.informs.org/doi/10.1287/trsc.1120.0445.

[Ras26d] Florian Rascoussier. *Routing numerical contracts: code, input panels and results*. Public code, compact input panels and numerical evidence. 2026/10/14. URL: https://github.com/0nyr/routing-numerical-contracts.

[Gol91] David Goldberg. "What Every Computer Scientist Should Know About Floating-Point Arithmetic". In: *ACM Computing Surveys* 23.1 (1991), pp. 5–48. DOI: 10.1145/103162.103163. URL: https://docs.oracle.com/cd/E19957-01/816-2464/ncg_goldberg.html.

[Ras+26] Florian Rascoussier, Romain Billot, Lina Fahed, and Christine Solnon. "Impact of Scaling and Rounding on Metaheuristic Performance for the Vehicle Routing Problem with Time Windows". In: *16th Metaheuristics International Conference (MIC)*. Ischia, Italy: Springer, 2026. arXiv: 2606.09196. URL: https://hal.science/hal-05646952.

[Wou24] Niels Wouda. *Switch back to doubles: PyVRP issue 623*. 2024/06/26. URL: https://github.com/PyVRP/PyVRP/issues/623 (visited on 2026/09/30).

[Wou26] Niels Wouda. *Switch back to floats: PyVRP pull request 1196*. Merged on 30 September 2026. 2026/09/27. URL: https://github.com/PyVRP/PyVRP/pull/1196 (visited on 2026/09/30).

[ZS22] Raffaele Zippo and Giovanni Stea. "Nancy: An efficient parallel Network Calculus library". In: *SoftwareX* 19 (2022), p. 101178. DOI: 10.1016/j.softx.2022.101178. URL: https://arxiv.org/abs/2205.11449v2.

[ZNS23] Raffaele Zippo, Paul Nikolaus, and Giovanni Stea. "Extending the network calculus algorithmic toolbox for ultimately pseudo-periodic functions: pseudo-inverse and composition". In: *Discrete Event Dynamic Systems* 33 (2023), pp. 181–219. DOI: 10.1007/s10626-022-00373-5.

[MPF25] MPFR Project. *GNU MPFR 4.2.2 Manual*. 2025. URL: https://www.mpfr.org/mpfr-current/mpfr.html (visited on 2026/10/14).

[Joh17] Fredrik Johansson. "Arb: Efficient Arbitrary-Precision Midpoint-Radius Interval Arithmetic". In: *IEEE Transactions on Computers* 66.8 (2017), pp. 1281–1292. DOI: 10.1109/TC.2017.2690633. arXiv: 1611.02831.

[Pyt26c] Python Software Foundation. *Built-in functions: sum*. 2026. URL: https://docs.python.org/3/library/functions.html#sum (visited on 2026/09/30).

[VS20] Thomas R. Visser and Remy Spliet. "Efficient Move Evaluations for Time-Dependent Vehicle Routing Problems". In: *Transportation Science* 54.4 (2020/07), pp. 1091–1112. DOI: 10.1287/trsc.2019.0938. URL: https://pubsonline.informs.org/doi/10.1287/trsc.2019.0938.

[Bla+24] Jannis Blauth, Stephan Held, Dirk Müller, Niklas Schlomberg, Vera Traub, Thorben Tröbst, et al. "Vehicle routing with time-dependent travel times: Theory, practice, and benchmarks". In: *Discrete Optimization* 53 (2024/08), p. 100848. DOI: 10.1016/j.disopt.2024.100848. URL: https://linkinghub.elsevier.com/retrieve/pii/S1572528624000276.

[DRV24] Said Dabia, Stefan Ropke, and Tom Van Woensel. "Erratum: "Branch and Price for the Time-Dependent Vehicle Routing Problem with Time Windows"". In: *Transportation Science* 58.5 (2024/09), pp. 919–924. DOI: 10.1287/trsc.2024.erratum.v58.n5. URL: https://pubsonline.informs.org/doi/10.1287/trsc.2024.erratum.v58.n5.

[Ras26e] Florian Rascoussier. *Thread Scaling of Hexaly on the TDVRPTW across Two Model Encodings. An Experimental Report: Binding Choice, Slice-Count Choice, and Two Thread Ladders*. Version 2. 2026. arXiv: 2608.10079. URL: https://arxiv.org/abs/2608.10079.

A The 2 time-dependent recipes

This section gives the route-level pseudocode and the numerical primitives that distinguish the releases. For a customer j with arrival u, service s_j and window $[e_j, l_j]$, the mathematical ready-time map is $\theta_j(u) = \max(u, e_j) + s_j$, defined only when $u \le l_j$. An Arrival-Time Function (ATF) α_{ij} maps departure from i to arrival at j. Depot waiting applies to the departure domain, while the final return is cut at the depot due date without a waiting clamp. A route duration is final arrival minus its depot departure.

We store a nondecreasing breakpoint chain (x_k, y_k). At a repeated abscissa, evaluation selects its first ordinate. Between distinct abscissae it interpolates between the enclosing points. Monotone emission clamps a newly computed coordinate below its preceding emitted coordinate and drops exact consecutive duplicates. The release does not normalize collinear breakpoints. These choices, and the event ordering in Algorithm 1, belong to the recipe. The pseudocode summarizes the frozen source, which remains the definitive executable specification [ANR26b; ANR26c].

A.1 Generic composition and interpolation

For strictly interior x, the original forward interpolation and inverse interpolation use

$$q = \mathrm{fl}\left(\frac{\mathrm{fl}(x - x_a)}{\mathrm{fl}(x_b - x_a)}\right), \qquad y = \mathrm{fl}(y_a + \mathrm{fl}(q\ \mathrm{fl}(y_b - y_a))), \tag{1}$$

$$p = \mathrm{fl}\left(\frac{\mathrm{fl}(u - y_a)}{\mathrm{fl}(y_b - y_a)}\right), \qquad x^* = \mathrm{fl}(x_a + \mathrm{fl}(p\ \mathrm{fl}(x_b - x_a))). \tag{2}$$

Every arithmetic operation shown has its own binary64 rounding. On a vertical piece, inverse interpolation gives the shared abscissa. At an exact knot, evaluation uses the stored ordinate rather than recomputing it.

For an inner plateau tied to an outer jump, this emission order preserves the lower outer ordinate across the plateau and emits the remaining vertical run only at its last abscissa. It is not a Cartesian product of the tied points. The arithmetic mode affects both forward interpolation and preimage calculation, so a changed ordinate can affect later event positions and branch selection.

A.2 Release 0.11.0: generic sequential folding

The original checker starts with an identity chain on the intersection of the ATF horizon and depot departure window. For each customer it composes an arc and then a ready-time chain. With a window, the latter has abscissae $[0, e_j, l_j]$ and ordinates $[e_j + s_j, e_j + s_j, l_j + s_j]$, with exact duplicates removed. Without a window it uses a service chain on $[0, M]$, where M is the current accumulator's maximum image. Algorithm 2 includes the return cut and earliest-minimizer rule [ANR26b].

The generic representation of $u + s$ inside a window can calculate

$$\mathrm{fl}\left((e + s) + \mathrm{fl}\left(\mathrm{fl}\left(\frac{u - e}{l - e}\right)\mathrm{fl}((l + s) - (e + s))\right)\right), \tag{3}$$

with each inner arithmetic operation also rounded. For $u = 1009$, $e = 250$, $l = 12360$, $s = 5$, the reproduced result is 1014.0000000000001. The direct map gives 1014. This is a recipe difference on the same operands, even before any large discontinuity amplification occurs.

A.3 Release 0.12.0: restriction, direct ready times and a slope shortcut

The newer checker replaces composition with the initial identity by direct restriction of the first ATF. Interior restriction endpoints still need interpolation. It replaces each ready-time composition and

Algorithm 1 Generic event-merge composition $H = F \circ G$

Require: Nondecreasing chains F, G and interpolation mode m

Return empty if either chain is empty
$a \leftarrow \max(x_0^F, y_0^G)$, $b \leftarrow \min(x_{last}^F, y_{last}^G)$
Return empty if $a > b$
$i \leftarrow$ first F knot with $x_i^F \geq a$, $j \leftarrow$ first G knot with $y_j^G \geq a$
$H \leftarrow$ empty chain
while an unprocessed F abscissa or G ordinate is at most b **do**
    $u \leftarrow$ smaller available event value (including equal values)
    $Y \leftarrow$ every consecutive F ordinate whose abscissa is exactly u, advance i
    $X \leftarrow$ every consecutive G abscissa whose ordinate is exactly u, advance j
    if Y is empty **then**
        $Y \leftarrow [\mathrm{Interp}_m(F_{i-1}, F_i, u)]$
    end if
    if X is empty **then**
        $X \leftarrow [\mathrm{Cross}_m(G_{j-1}, G_j, u)]$
    end if
    for x in X, in order **do**
        $\mathrm{Emit}(H, x, Y_0)$
    end for
    for each remaining y in Y, in order **do**
        $\mathrm{Emit}(H, X_{last}, y)$
    end for
end while
return H

Algorithm 2 Route evaluation in 0.11.0 (`td-fold/1`)

Require: Route $r = (v_1, \ldots, v_n)$ and fixed canonical input

$[L, U] \leftarrow$ horizon intersected with the depot departure window, if present
Return infeasible if $L > U$
$A \leftarrow \mathrm{Id}_{[L,U]}$, $i \leftarrow 0$
for j in r **do**
    $A \leftarrow \mathrm{Compose}_{generic}(\alpha_{ij}, A)$
    Return infeasible if A is empty
    Construct θ_j from the window/service, or service chain on $[0, \max \mathrm{img}(A)]$
    $A \leftarrow \mathrm{Compose}_{generic}(\theta_j, A)$
    Return infeasible if A is empty
    $i \leftarrow j$
end for
$A \leftarrow \mathrm{Compose}_{generic}(\alpha_{i0}, A)$
if depot window exists and A is nonempty **then**
    $A \leftarrow \mathrm{Compose}_{generic}(\mathrm{Id}_{[0,l_0]}, A)$
end if
Return infeasible if A is empty
Scan $\mathrm{fl}(y_k - x_k)$ in knot order, update only on strict improvement
return minimum duration and earliest winning knot abscissa

the return cut by a specialized scan of the accumulator. Algorithm 3 shows the fold and Algorithm 4 shows that specialized scan [ANR26c].

Algorithm 3 Route evaluation in 0.12.0 (`td-fold/2`)

Require: Nonempty route $r = (v_1, \ldots, v_n)$ and fixed canonical input
$[L, U] \leftarrow$ horizon intersected with the depot departure window, if present
Return infeasible if $L > U$
$A \leftarrow \mathrm{Restrict}_{slope}(\alpha_{0v_1}, [L, U])$, $i \leftarrow 0$
for j in r **do**
    if j is not the first customer **then**
        $A \leftarrow \mathrm{Compose}_{slope}(\alpha_{ij}, A)$
    end if
    Return infeasible if A is empty
    $A \leftarrow \mathrm{Ready}_{slope}(A, e_j \text{ or none}, l_j \text{ or none}, s_j)$
    Return infeasible if A is empty
    $i \leftarrow j$
end for
$A \leftarrow \mathrm{Compose}_{slope}(\alpha_{i0}, A)$
if depot window exists and A is nonempty **then**
    $A \leftarrow \mathrm{Ready}_{slope}(A, \mathrm{none}, l_0, 0)$
end if
Return infeasible if A is empty
Scan $\mathrm{fl}(y_k - x_k)$ in knot order, update only on strict improvement
return minimum duration and earliest winning knot abscissa

Algorithm 4 Specialized ready-time scan used by 0.12.0

Require: Accumulator A, optional earliest e, optional latest l, service s
Return empty if A is empty, or if l exists and $y_0 > l$
$H \leftarrow$ empty chain
for each knot k, in order **do**
    if $k > 0$, e exists and $y_{k-1} < e < y_k$ **then**
        $\mathrm{Emit}(H, \mathrm{Cross}_{slope}(A_{k-1}, A_k, e), \mathrm{fl}(e + s))$
    end if
    if $k > 0$, l exists and $y_k > l$ **then**
        if $y_{k-1} < l$ **then**
            $\mathrm{Emit}(H, \mathrm{Cross}_{slope}(A_{k-1}, A_k, l), \mathrm{fl}(l + s))$
        end if
        Stop the scan
    end if
    $u \leftarrow \max(y_k, e)$ if e exists, otherwise y_k
    $\mathrm{Emit}(H, x_k, \mathrm{fl}(u + s))$
end for
return H

The slope mode tests the rounded differences $\mathrm{fl}(y_b - y_a) = \mathrm{fl}(x_b - x_a)$. On a match, forward interpolation returns $\mathrm{fl}(y_a + \mathrm{fl}(x - x_a))$, and inverse interpolation returns $\mathrm{fl}(x_a + \mathrm{fl}(u - y_a))$. Otherwise it uses the generic formula. Both releases retain binary64 arithmetic, exact comparisons, the generic event merge for arc composition and a left-to-right route fold. The newer release adds specialized primitives and new branches, so it is still implementable as a portable recipe, provided these are specified and tested.

For both contracts, structural validity checks require customer partition, capacity and any declared fleet limit. The solution total starts at 0.0, adds the route durations in ascending first-customer

order and, for FleetCostDuration, performs one multiplication by the number of routes and one final addition of the vehicle fixed cost. A declared total is checked by exact equality. The first-customer keys are unique in a valid elementary route partition. Empty routes are not supported by the solution models.

A.4 A qualified exactness claim

Direct ready-time application removes a specific interpolation of a mathematically unit-slope map. On integer breakpoints/windows/services with slopes 0, 1 or vertical pieces, the newer recipe can coincide with the exact model when every required intermediate sum, difference, preimage and duration remains exactly representable. A sufficient guarded regime is integer operations within the binary64 exact-integer range, together with valid finite domains and the point convention stated above. This condition must cover intermediate values, not only the input integers.

A general statement that the newer fold is exact on arbitrary stored doubles is false. Generic non-unit interpolation remains rounded, the emitted chain can still be clamped after roundoff, and the unit-slope predicate itself is a rounded comparison. Small dyadic inputs are exact operands but do not guarantee exact differences or sums. The new tests on an integer stepwise class support that class and do not establish unrestricted mathematical exactness.

B Independent exact models and measured alternatives

This section presents a reference algorithm that does not reuse the checker composition and distinguishes the alternatives we implemented from those we only analyse. We choose rational arithmetic because every finite stored breakpoint can be represented exactly. We do not assume that arbitrary real-valued input or unrounded Euclidean geometry has a finite rational representation.

B.1 Forward partitioning instead of composition

Our oracle represents each arc as point evaluations and affine open intervals. A ready-time stage checks the due date, applies waiting and adds service exactly. Starting from the departure endpoints, it pulls the boundaries of each next stage back through every affine interval of the existing prefix. Its numerical state consists of exact rationals and a cut set, not a rounded output breakpoint chain. Algorithm 5 states the procedure.

The induction behind this partition is simple: once all previous stage-boundary preimages are cuts, the prefix is affine or infeasible on each remaining open interval. A constant prefix requires no additional interior split. Each next-stage boundary can then be pulled back exactly through the nonconstant affine prefix. After the last stage, duration is affine on each feasible open interval, so the least possible value is at a point or an endpoint limit. We check endpoint membership separately. In a more general domain convention a limit can be an unattained infimum, which must not be reported as an attained optimum. This algorithm is intentionally structurally different from the checker's event merge.

The reference is still an implementation requiring validation. We test hand-computed jump, singleton, empty-domain and non-unit examples, an unattained infimum, and 80 independent exhaustive integer-grid comparisons on a small slope-0/1 class. Those checks complement the reasoning above. They are not a machine-checked proof of every Python branch. We impose a 120-second route/interpretation budget and a 300-second solution budget, record any exceeded budget explicitly, and measure cut counts and numerator/denominator bit lengths. No reference in the completed panels reached these limits.

Algorithm 5 Independent rational route optimization

Require: Rational stages $f_1, \ldots, f_m$ and departure interval $[L, U]$

$C \leftarrow \{L, U\}$ if $L \le U$, otherwise empty, $P \leftarrow \mathrm{Id}$
for stage f_j **do**
    for each open interval (a, b) between consecutive cuts in C **do**
        Evaluate P exactly at $(2a + b)/3$ and $(a + 2b)/3$
        Skip this interval if the prefix is infeasible or constant there
        Recover its exact affine slope and preimages of every boundary of f_j
        Insert preimages strictly inside (a, b) into the next cut set
    end for
    $C \leftarrow$ updated cut set, append f_j to the forward prefix
end for
Evaluate the full forward route exactly at each cut
Evaluate the affine duration limits at both ends of every feasible open interval
Select the least duration, distinguish attainment and return the earliest attained minimizer
return infeasible if there is no feasible point or open interval

B.2 Isolated changes and a stricter slope predicate

We evaluate 3 isolated modifications of the original recipe: direct first-arc restriction, direct vertex/return transforms, and the slope shortcut alone. We also evaluate the complete newer recipe. These are diagnostic contracts, not published library releases. They show that the contribution of a change can depend on the surrounding arithmetic and that numerical fidelity need not improve monotonically when branches are combined.

We additionally implement a stricter version of the slope predicate. For the reproduced segment

$$(x_a, y_a) = (0, 2^{-51}), \qquad (x_b, y_b) = (5, 5), \qquad x = 0.011393229166666666, \tag{4}$$

the rounded rise equals the run, although the exact binary-operand slope is $1 - 2^{-51}/5$. Generic interpolation returns 0.011393229166667108, equal to the correctly rounded rational result. The newer shortcut returns 0.01139322916666711. This is a primitive-level counterexample to general mathematical exactness and to a guarantee of universal fidelity improvement. It is not evidence of a large optimal-route error on a canonical benchmark.

The stricter predicate uses an error-free sum transformation on the subtraction operands. If $d = \mathrm{fl}(a - b)$, it obtains a residual e such that $a - b = d + e$ under round-to-nearest binary64 without overflow and with the prescribed separate operations. This is the TwoSum transformation applied to $a + (-b)$ [ORO05]. It requires equality of both rounded differences and their residuals. The core operations are:

```
d = a - b
z = d - a
residual = (a - (d - z)) + ((-b) - z)
unit = finite(rise) and finite(run) and rise == run
unit = unit and rise_residual == run_residual
```

The prototype passes 2,000 rational predicate comparisons and rejects the counterexample's shortcut. It retains the generic fallback and both direct transforms, so it still has rounded non-unit interpolation. Its extra operations and its rounding/underflow assumptions must become part of a contract if adopted. We do not treat it as a universal correction or as a production recommendation without broader conformance tests.

B.3 Alternatives analysed but not installed

A rational authority can compare exact costs independently of summation order. It must specify whether it ranks exact values, rounds the final value once to binary64, and returns an exact departure or a representable executable departure. Rounding an exact rational minimizer to a double can cross a deadline or jump. A nearest-double cost likewise does not specify how a double-based solver must price moves or bounds. An exact reference is therefore valuable even if it is not selected as the operational authority.

Scaled integers can remove rounding in additions when all values lie on a declared scale and every intermediate value fits the integer range. They change the input model when values are rounded onto that scale. They do not in general close piecewise-affine interpolation under integer arithmetic, since preimages can be nonintegral. For signed 64-bit arithmetic, all scaled arc costs, route accumulations, fixed costs, search penalties and sentinel values need a range argument. For example, 10^4 arcs of size 10^{12} at scale 10^6 require an aggregate of 10^{22}, exceeding $2^{63} - 1$. We evaluate this range analytically and do not execute signed integer overflow in C++.

A fixed higher precision reduces many roundoff errors but still requires input semantics, operation order and branch rules. It cannot guarantee exact jump-side or equality decisions. Adaptive precision with interval enclosures can certify a strict comparison once intervals separate. Exact ties, uncertain membership at jumps and overlapping objective intervals need escalation or an explicit unresolved outcome. An epsilon shifts the model's hard boundaries and must have units and a declared meaning. Another option is to encode segment intent explicitly, for example a declared unit-slope segment or constant travel time, rather than infer that intent from rounded endpoint differences. This distinguishes the intended generating model from exact interpolation through rounded endpoints. It requires a versioned artifact representation and reader/solver migration. We analyse it conceptually without implementing a new data format.

We discuss these choices theoretically in this phase. We do not implement an interval/adaptive replacement checker.

C Experimental protocol and coverage

This section states the panels and reproducibility checks before interpreting the empirical comparisons. We evaluate fixed routes and stored route sets. No experiment searches for a new optimum, establishes a new lower bound or renews a historical certificate. We preserve the canonical ATFs and static costs rather than re-deriving instance distances.

The original panel uses the 146-entry upstream solution file, containing 898 routes. We remove its duplicated end depot explicitly and evaluate the common canonical Dabia2013 artifacts. All route folds are compared with both rational interpretations and all isolated variants. Original published departure times are separate historical witnesses. Their departure policy, solver tolerances and historical arithmetic need not equal the minimizing departure of either canonical fold.

The broader exact panel contains 35 selected BKS records and 243 routes. Selection uses the minimum SHA-256 of a fixed seed plus relative path within each family/problem/customer-count stratum of at most 100 customers, excluding the fully evaluated original Dabia2013 panel. The recorded selection covers Ari2018, Vu2020, Rifki2020, Poryos2026 and the available small Blauth2024 record. We do not infer large-instance rational tractability from this panel.

The synthetic panel has 168 fixtures, with a fixed seed and explicit jump, plateau, deadline-neighbor, nonzero-horizon and non-unit examples. The integer-generated class deliberately targets known rounding amplification. It is not a random sample of practical routing instances, and its disagreement frequency has no population interpretation. We separately reproduce the upstream Rifki-30 route that motivated the change.

The full store enumeration has 1976 static BKS records and 3072 TD BKS records. We successfully evaluate 1956 static and 3052 TD records. The 20 static omissions are large Mamut2026 fastest/shortest distance sidecars, and the 20 TD omissions are recorded missing sidecars. The machine-readable manifests contain their exact identifiers and errors. Availability is neither validity nor infeasibility. Counts below always refer to successfully materialized inputs [Ras26d].

We verify that the TD harness reproduces both complete frozen tagged packages on 1,066 routes (898 original plus 168 synthetic), with no mismatch in feasibility, duration or departure. We also compare the unchanged static checker loop structurally between tags and check its outputs against our independent static arithmetic on every available record. The entire original TD result digest is identical under CPython 3.11.15, 3.12.13, 3.13.13 and 3.14.4 on this Linux x86-64 host. This is not an architecture-wide conformance certificate. The C++ diagnostic reproduces selected expressions with GCC 15.2.0, not a full independent route engine.

Timing fields include Python arc-container construction and run concurrently with other work. Worker maximum resident memory is cumulative. We use them to expose the resource profile of this implementation, not to claim solver speed-ups or a production performance ranking. The protocol was recorded during the investigation and was not externally preregistered.

D Original Dabia2013 solutions

This section compares the original routes against a common artifact snapshot. Both operational contracts accept all 898 routes and all 146 route sets, and the structural partition/capacity/fleet checks pass. Their solution totals differ for 31 records, while each matches 48 published totals exactly. Exact reproduction of the historical published value is therefore informative but does not select a winner. Appendix I reports every solution total.

Table 2 separates correctly rounded agreement with the exact binary-operand route optimum from forward-departure diagnostics. A correctly rounded optimum match does not imply exact rational equality, and a composed route's temporal validity does not imply that its selected binary64 departure survives a different forward recipe.

Table 2: Original panel: 898 routes. Reference agreement uses nearest-binary64 rounding of the independent rational optimum. Forward diagnostics use generic arc interpolation, direct waiting/service and exact due-date comparisons. The last diagnostic column includes both rejection and accepted schedules with a different duration.

Recipe	Rounded optimum matches	Forward rejections	Cost mismatch or rejection	Maximum absolute error
v011	455	30	154	7.79e-12
restriction	452	27	151	7.79e-12
vertex	485	25	65	7.79e-12
slope	494	25	77	7.79e-12
v012	494	25	77	7.79e-12
strict	485	21	63	7.79e-12

The old recipe has 455 correctly rounded rational-optimum matches, compared with 494 for the complete newer recipe. The maximum absolute route-value discrepancy is approximately 7.79×10^{-12} for both. Direct vertex transforms alone improve several measurements, and combining them with the slope rule does not minimize every forward diagnostic count. The 3 isolated variants therefore support attributing effects rather than treating the release as one undifferentiated numerical improvement.

Generic forward replay rejects the selected departure for 30 old-fold routes and 25 new-fold routes. Including cost disagreements among accepted schedules gives 154 and 77 discrepancies. Slope-aware forward replay under the newer arithmetic gives 25 rejections and 54 mismatches-or-rejections.

These are different algorithms: the historical generic arc evaluator is not implicitly replaced by a slope-aware one just because composition changed. A future contract should explicitly identify the relationship between its composed function, its reported cost and its executable departure witness.

Exact forward replay of the binary64 departures under the binary-rational model rejects 148 old-fold departures and 140 new-fold departures. This stricter diagnostic concerns the exact finite-input mathematical model at a particular rounded departure. It does not say that 148 or 140 whole routes have no feasible departure. Similarly, generic replay of the original published departures rejects 22 routes, and exact binary-operand replay rejects 111. The result is consistent with the need to retain the historical computation and conventions of those witnesses rather than rewriting their validity status from one alternative evaluator.

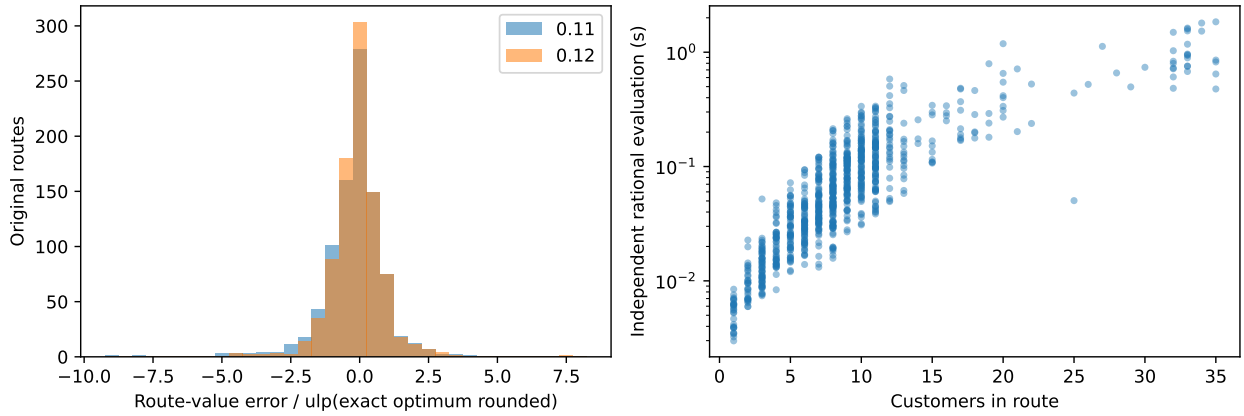


Figure 1: Original route-value error relative to the exact binary-operand optimum, in units of the binary64 spacing at its rounded value (left), and the independent rational evaluator's descriptive wall time against route length (right). Timings include the implementation described in Section C and do not measure KAYROS performance.

The rational reference completes every original route under both input interpretations. The binary-operand and shortest-decimal models have different exact optimum values on 895 of the 898 routes, with different rounded values on 377. Their maximum absolute optimum difference is approximately 7.68×10^{-12}, and they agree on whole-route feasibility here. Thus choosing a rational implementation still requires choosing the input model. The largest recorded cut numerator/denominator length is 954 bits. These bit lengths concern the cut set, not every intermediate ordinate, and are a lower view of the implementation's exact arithmetic sizes. They demonstrate why rational arithmetic is more than a switch from double to a fixed-width integer, even on moderate route lengths.

E Stepwise examples and the broader TD store

This section distinguishes a reproduced material step effect from the broader corpus measurements. For the published Rifki-30 regression route $[23, 21, 12, 3, 4, 25, 30, 29]$, the old fold returns duration 1768 at departure 1301, and the newer fold returns 1743 at departure 1341. The independent rational optimizer returns $(1743, 1341)$ on the frozen binary operands. This establishes improved fidelity on this fixed route and operational incompatibility between the contracts. It does not establish a new optimum over route sets.

A jump can amplify a small upstream time perturbation into a large cost change: if the arrival curve jumps by J at u, the values at u and just to its right can differ by nearly J. Without continuity or a local margin away from every event, a small interpolation-error bound gives no small downstream cost bound. A hard due date can turn the same branch change into rejection. The original computational recipe is still well-defined, but this is a substantive limitation when

interpreted as a model of ideal feasible schedules.

Table 3: Available canonical TD BKS records by normative family label. "Changed" compares old/new values on the same current routes and artifacts. "Feasibility flips" compares their nonempty-domain route-set verdicts.

Family	Available records	Changed totals	Feasibility flips
Ari2018	320	52	0
Blauth2024	20	0	0
Dabia2013	336	113	0
Lera2026	960	278	0
Poryos2026	720	202	0
Rifki2020	360	170	0
Vu2020	336	56	0

The full available corpus has 871 old/new total differences and 0 temporal feasibility flips on the current route sets. The largest absolute total difference is 25. All available current records pass the structural checks and match their declared values under the newer recipe. Those counts differ in meaning from the campaign's cost-edit count, since the campaign also changed routes and left some declared values numerically unchanged. The 42 changed TD twin route sets improve their target-instance values under both frozen contracts. This comparison establishes a fixed-route improvement in the available target artifacts, not a proof of the full relaxation relationship or renewed optimality.

The broader exact panel completes all 243 routes in both interpretations, with no whole-route feasibility disagreement on that selection. Its largest cut bit length is 1,503 and largest cut count is 1,828. Table 4 reports the isolated variants. Binary-operand and shortest-decimal exact optimum values differ on 167 routes, with different rounded values on 118 and maximum absolute difference approximately 2.36×10^{-11}. Larger exact instances and solver pricing campaigns remain outside the completed panel.

Table 4: Broader exact panel: 243 routes from 35 recorded strata. Rounded optimum agreement uses the binary-operand reference. Forward replay uses generic arc interpolation. Median fold times are descriptive wall times with Python setup included, measured during separate experiment runs with varying load. They are not a controlled comparison of production throughput.

Recipe	Rounded optimum matches	Forward rejections	Maximum absolute error	Median fold seconds
v011	151	3	8.34e-09	0.00312
restriction	157	2	8.34e-09	0.00289
vertex	169	3	8.34e-09	0.00263
slope	175	2	8.34e-09	0.00295
v012	175	2	8.34e-09	0.00247
strict	174	2	8.34e-09	0.00124

The independent binary-rational evaluator's median wall time is approximately 0.058 seconds per original route and 0.709 seconds per broader-panel route. The longest broader-panel binary evaluation takes approximately 53.64 seconds. The recorded float fold medians are below 0.001 seconds on the original panel and below 0.004 seconds on the broader panel, including setup. These observations support the oracle's use for verification at the measured sizes. They do not provide a controlled solver speed ratio or establish the cost of an optimized exact implementation.

The 168 targeted synthetic cases produce 1 whole-route feasibility disagreement and 11 additional exact-value disagreements for the original recipe, compared with 0 and 1 for the newer recipe. These count exact equality to the rational result, unlike the rounded-agreement tables. The class

deliberately concentrates on known numerical mechanisms, so its disagreement frequencies should not be treated as empirical rates for a benchmark population.

The strict slope prototype is evaluated against the already computed independent references. Its purpose is to isolate the rounded guard's semantics. On the original panel it has 485 correctly rounded optimum matches and 21 generic-forward rejections, versus 494 and 25 for the published newer recipe. On the broader panel those optimum-match counts are 174 and 175. Thus a stricter primitive predicate can improve one criterion without improving every aggregate measure. It improves the primitive counterexample, while its effects on the measured solution panels are reported alongside the other variants. We do not extrapolate the targeted improvement to every canonical route or declare the prototype a superior normative checker from those counts alone.

F The static checker and incumbent ranking

This section gives static routing equal attention because its arithmetic and policy changes are distinct from the TD fold. The ordinary checker loop is unchanged between releases: it traverses the submitted routes, adds every depot/customer arc into one accumulator and evaluates static time windows by earliest forward traversal. It starts time at 0, adds travel, waits to ready time, compares exactly to due time, then adds service. At the return depot it adds travel and checks the depot window. We reproduce those operations rather than silently imposing a new departure policy.

Algorithm 6 Static pricing and time-window validation in both releases

$C \leftarrow 0$, served set $\leftarrow \varnothing$
for route r in submitted order **do**
    $i \leftarrow 0$, load $\leftarrow 0$, $t \leftarrow 0$
    for customer j in r **do**
        Reject invalid/repeated customer or capacity overflow
        $C \leftarrow C + c_{ij}$
        if time windows exist **then**
            $u \leftarrow \max(t + c_{ij}, e_j)$, reject if $u > l_j$, $t \leftarrow u + s_j$
        end if
        $i \leftarrow j$
    end for
    $C \leftarrow C + c_{i0}$
    If windows exist, reject if $\max(t + c_{i0}, e_0) > l_0$
end for
Reject missing customers or exceeded fleet limit
Reject a declared cost unequal to C, otherwise return C

For integer costs, Python integers remain exact. For floating costs, the loop is a flat sequential fold, without a per-route reset of the cost accumulator. The static helper `compute_solution_cost` instead computes each route subtotal and calls built-in `sum()` on those subtotals. Parenthesization can therefore differ from the checker even in one interpreter. Moreover, Python 3.12 changed the float `sum()` algorithm. Our positive witness $[10^{16}, 1, 1]$ returns 10^{16} under 3.11's `sum()`, and $10^{16} + 2$ under the tested 3.12–3.14 interpreters. The explicit left fold returns 10^{16} throughout. This is separate from the unchanged TD explicit total [Pyt26c].

The new static BKS store validates candidates, canonicalizes their route ordering for storage, and compares exact rational sums of `repr(float(arc_cost))`. It still stores the ordinary checker value. Thus the ranking objective and the displayed/stored computational value are distinct. Exact decimal ranking removes permutation ties and, on symmetric matrices, reversal effects in the mathematical sum. It can be a sensible declared policy. It nevertheless changes the meaning of a strict incumbent improvement relative to binary64 checker scoring [ANR26c].

Table 5: Static sweep, available records only. Order sensitivity tests the submitted, canonical, reversed route-list and fixed-seed shuffled orders, with every route's internal direction unchanged. "Grouped" compares the helper's grouped accumulation with the flat checker in Python 3.13. "Scale 1000" requires every selected decimal-scaled arc and their total to be integral and within nonnegative signed 64-bit range.

Family	Available	Order-sensitive	Grouped differs	Scale 1000
Dimacs2021	468	0	0	468
Mamut2026	310	310	271	310
Ortec2022	350	0	0	350
Poryos2026	360	330	267	360
Sintef2008	468	429	386	0

Every one of the 1956 available static records passes the tagged structural/time-window checker and matches its declared cost. The tested permutations produce differing totals for 1069 records, and the grouped helper differs for 924. Neither phenomenon alone proves that a prescribed flat-checker computation is wrong. It does show that routing the same solution through an alternative helper or reduction order can break exact comparisons. The independent binary-rational and decimal-rational temporal traversals do not flip feasibility for any of these available stored route sets. The constructed $0.1 + 0.2$ at deadline 0.3 example does distinguish decimal-exact from binary64/binary-rational feasibility, so absence of flips in this corpus is not a general equivalence claim.

The 31 restored Mamut2026 records are exact decimal ties on their route arc sums. Under canonical flat binary64 scoring, 11 restored current route sets are worse than their campaign predecessors, 4 are better and 16 tie. The separate Poryos static twin replacement improves both metrics. These observations establish that the audit's tie label is policy-dependent. They do not establish that all 31 should be restored or removed under every possible policy. A hierarchical objective also compares route count before its secondary cost, and that tier must remain explicit.

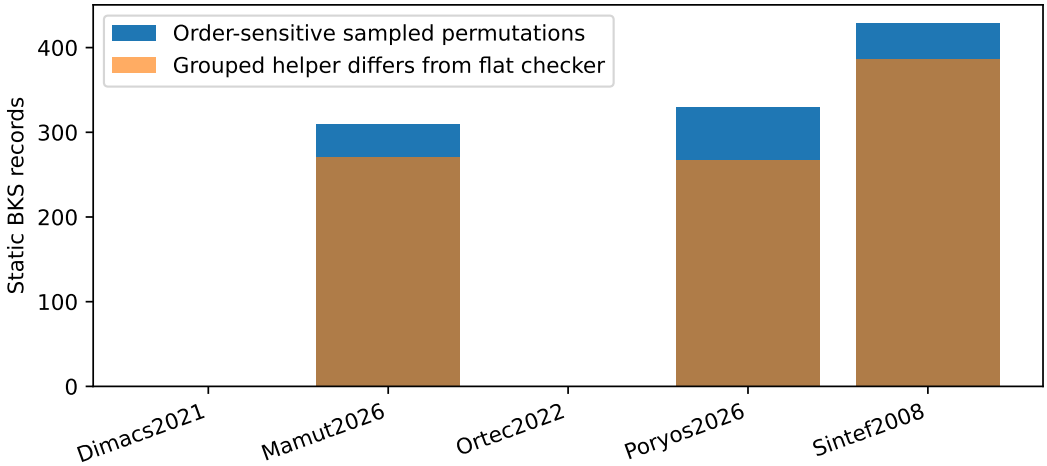


Figure 2: Static ordering and grouped-helper effects in the available store. These are measurements of different computational reductions on fixed route sets, not changes to their exact arc multisets.

The scale-1000 eligibility test passes for 1488 records. It fails for the full-precision Sintef2008 costs, whose decimal spellings generally require finer scales. This test checks every selected route arc and the complete arc-sum objective, not every matrix entry, solver penalty, service time or intermediate arithmetic. An integral total alone would be insufficient, since fractions of a scaled unit can cancel in a sum. The `fsum` experiment offers a more accurate reduction of binary operands, but is a different cost recipe and inherits platform caveats documented by Python. A correctly rounded sum of exact

binary operands and an exact sum of decimal spellings are also different proposals [Pyt26b].

The submitted flat total matches the correctly rounded exact binary-operand sum on 1,037 of 1,956 records. Canonical flat ordering gives 1,022 matches, grouped route reduction gives 1,672, and `fsum` gives 1,956 on this host and panel. Canonical ordering therefore provides deterministic storage order, without guaranteeing greater summation accuracy. The rounded exact binary and decimal sums differ on 39 records, further distinguishing a binary-operand reduction proposal from the newer decimal ranking policy.

G Consequences of the September change campaign

This section separates numerical migration effects from independent integrity repairs. The pinned diff contains 928 BKS cost edits, of which 870 concern TD records and 58 static records, and 74 route-list edits. Of the latter, 43 are twin replacements (42 TD and 1 static) and 31 are the Mamut restorations discussed above. The original audit ledger's numerical claims need to be interpreted under their chosen ranking/feasibility metric, rather than treated as contract-neutral descriptions.

We independently confirm 210 changed optimality metadata records. Their changed keys are `note` and `proven_optimum`. The notes acknowledge repricing from `td-fold/1` to `td-fold/2`, leave the prover's dual bound as a historical value and state that recertification is pending. The `proven` flag is retained. This creates mixed metadata if read without its note: a repriced incumbent value appears next to an older-contract proof claim. We should preserve the original evidence and attach the proof to its original contract, rather than erase it or infer a new proof from the repriced route set.

Twin relationships require their own care. Capacity-equivalent removal of a hard constraint preserves the usual exact-model containment, but a specialized computational recipe may evaluate two structurally related instance types through different operations. In particular, replacing waiting windows by an unrestricted service chain can itself change roundoff. We should test the claimed operational relaxation relationship under the chosen checker, even when the mathematical relaxation is sound. The newer objective-aware screen and depot-window check address real modeling issues, but cross-evaluated incumbents must be evaluated under each target's declared contract before any replacement policy is chosen.

The source review establishes separability and the documented reasons for these repairs. It is not a fresh security audit or an exhaustive verification of the tools' generation semantics. In particular, changing feasible anchors or generated time windows changes instance generation and should retain its generator/version provenance, even if the algorithm fixes a practical failure. We do not roll those changes into a numerical-checker conclusion.

We identify a concrete cascading compatibility issue in the tools' legacy run handling: schema-1 runs are revalidated under the installed checker, but a changed verdict or cost is described as instance regeneration. Its cached legacy validation is keyed by instance hash without a checker-contract identifier. The newer schema records instance bytes but does not itself identify the evaluation contract. Hence unchanged instance bytes can coexist with a changed computational interpretation, and reverting or upgrading a checker can leave cached legacy status inappropriate. This should be handled as an explicit contract compatibility condition, preserving the original run and evaluator identity.

The reviewed staging repair is atomic per file with a recovery marker and roll-forward behavior, not one simultaneous atomic swap of an entire collection. The archive repair also still replaces an existing release-stamped subtree when fetching again, so its mitigation does not by itself preserve locally added BKS inside that subtree. These limits should remain visible when retaining the independent integrity repairs.

Table 6: Disposition classes for the 24 audit campaign resolutions. These are recommendations for separating a later repair/migration decision, not changes performed by this investigation. A sanitized source-path and disposition map accompanies the public evidence. The original audit ledger remains private.

Change group	Investigation conclusion
TD fold and repricing	Treat as a contract migration. Preserve both recipes, original declared values and original certificates. A fidelity gain on stepwise examples does not establish nonconformance to the old recipe.
Static decimal ranking and 31 restorations	Treat exact decimal ranking as a separate policy choice. Retain candidate validation and canonical storage as independently reviewable changes. Reassess incumbent choices using the adopted objective.
Twin replacements and objective-aware checks	Preserve candidates and cross-evaluation evidence. Validate target-specific timing, depot windows, objective and route constraints under each contract before selecting incumbents.
Sidecar hashes, atomic/staged writes and regeneration	These address provenance, stale data and partial publication independently of the choice of route arithmetic. We recommend retaining the repairs subject to their own regression evidence.
Instance identifiers, archive layout, exception handling and cost reporting	These address discovery and interface consistency. A blanket rollback would remove useful repairs. Keep them separable from numerical migration.
Export guards, site sorting, payload handling and download selection	These address mismatched exported instances and interface/integrity defects. Their retention does not require adopting `td-fold/2`.

H Literature and historical choices

This section locates the investigated recipes within the literature without attributing a finite arithmetic specification to papers that describe an exact mathematical operation. Visser et al. [VS20] use ready-time functions and composition for route evaluation and efficient neighborhood search. Their tree constructions exploit mathematical associativity. This does not establish bitwise associativity of a binary64 implementation. An authoritative sequential fold can therefore coexist with faster solver structures, provided accepted results return to the prescribed fold.

Blauth et al. [Bla+24] formulate continuous, nondecreasing piecewise-linear arrival-time functions and efficient operations, including scheduling. Their mathematical results provide a useful model and complexity context. They do not select the particular binary64 waiting-map interpolation or specialized arithmetic of either investigated release. Genuine jumps require an additional point/side convention. We preserve the canonical data's smallest ordinate at an exact repeated abscissa rather than assuming that a vertical chain connector is a continuum of possible arrival values.

Lera-Romero et al. [LMS20] investigate labeling algorithms with linear edge costs and discuss the importance of implementation choices to numerical experiments. Dabia et al. [DRV24] correct implementation results associated with the earlier branch-and-price work. These sources motivate careful validation, but do not establish that our original checker must be replaced by one particular arithmetic refinement. The 146 original solution records provide a concrete historical panel for our own comparison [Dab+13].

An adjacent literature demonstrates that exact rational piecewise-affine software is a realistic option. Zippo et al. [ZNS23] explicitly separate point elements and open segments when handling discontinuities. Zippo et al. [ZS22] implement rational arithmetic with arbitrary-size integers to

avoid overflow, with a finite-width alternative for performance. This supports investigating exact references and bit complexity. It does not imply that a rational implementation fits the throughput or integration requirements of KAYROS without a dedicated comparison.

Static routing has a related trade-off. Scaling and rounding alter the finite instance, as our earlier VRPTW study investigates. PyVRP's issue #623 discusses fractional cost modeling and integer range concerns, and its September 27, 2026 PR #1196 proposes returning to floats. The initial investigation captured the PR before its merge. It was merged on September 30, 2026, and we update its status here. We use this as evidence that the trade-off remains practical, without interpreting a project discussion as a universal performance result or a recommendation to copy its design [Ras+26; Wou24; Wou26].

I Original solution totals

This appendix records the original declared total and both contract totals for every original solution. Decimal strings are round-trip representations. The last column counts binary64 representable-value steps between the old and new totals, so small changes remain visible even when neighboring printed values look similar. These are fixed-route evaluations, not renewed optimality claims.

Table 7: All 146 original Lera-Romero solution totals.

Instance	Routes	Published	0.11	0.12	ULP steps
C101_100	10	97901.11368298682	97901.1136829868	97901.1136829868	0
C101_25	4	24709.17715754648	24709.177157546477	24709.17715754648	1
C101_50	6	49242.92185601275	49242.92185601275	49242.92185601275	0
C102_100	10	97662.59605672678	97662.59605672678	97662.59605672678	0
C102_25	3	24529.89265361813	24529.89265361813	24529.89265361813	0
C102_50	5	48545.67570963773	48545.675709637726	48545.675709637726	0
C103_100	10	97373.34559204272	97373.34559204272	97373.34559204272	0
C103_25	4	24395.91907624517	24395.919076245165	24395.91907624517	1
C103_50	5	48251.30012688959	48251.300126889575	48251.300126889575	0
C104_100	10	97224.07975000869	97224.07975000869	97224.07975000869	0
C104_25	3	24248.988513504446	24248.988513504446	24248.988513504446	0
C104_50	5	47982.62045686993	47982.62045686991	47982.620456869925	2
C105_100	10	97829.2465448466	97829.24654484661	97829.24654484661	0
C105_25	4	24697.510096595226	24697.510096595222	24697.510096595222	0
C105_50	5	48809.083908465866	48809.08390846585	48809.08390846585	0
C106_100	10	97692.51063072543	97692.51063072543	97692.51063072543	0
C106_25	4	24709.17715754648	24709.17715754648	24709.17715754648	0
C106_50	5	48679.42852469775	48679.428524697745	48679.428524697745	0
C107_100	10	97469.08339699972	97469.08339699972	97469.08339699972	0
C107_25	4	24524.793693584077	24524.793693584077	24524.793693584077	0
C107_50	5	48482.480282009754	48482.48028200975	48482.48028200975	0
C108_100	10	97213.7773015322	97213.77730153222	97213.77730153222	0
C108_25	4	24417.629368006114	24417.629368006114	24417.62936800612	2
C108_50	5	48219.24351422906	48219.24351422906	48219.24351422906	0
C109_100	10	97004.60303710586	97004.60303710586	97004.60303710586	0
C109_25	4	24337.612087928905	24337.6120879289	24337.61208792891	2
C109_50	5	48173.495150805844	48173.495150805844	48173.495150805844	0
C201_100	4	96115.821118694	96115.821118694	96115.821118694	0
C201_25	5	25606.02074228372	25606.020742283705	25606.020742283705	0
C201_50	5	49683.84758988022	49683.84758988022	49683.84758988022	0
C202_100	4	95960.13893527849	95960.13893527843	95960.13893527843	0
C202_25	3	24753.472986157954	24753.47298615795	24753.47298615795	0
C202_50	3	48727.408326543504	48727.408326543504	48727.4083265435	1
C203_25	2	24509.259321091573	24509.25932109157	24509.259321091566	1
C203_50	4	48482.24677094631	48482.2467709463	48482.2467709463	0
C204_25	2	24149.205676557634	24149.20567655762	24149.20567655762	0
C205_100	3	95763.66558507847	95763.66558507847	95763.66558507847	0
C205_25	4	25081.263407923005	25081.263407922994	25081.263407922994	0
C205_50	5	49130.718913917364	49130.718913917364	49130.718913917364	0
C206_100	3	95616.16600814607	95616.16600814607	95616.16600814607	0
C206_25	4	24953.470994079835	24953.47099407983	24953.47099407983	0
C206_50	5	49026.6201357225	49026.620135722485	49026.620135722485	0
C207_100	3	95613.10814392117	95613.10814392118	95613.10814392118	0
C207_25	3	24896.4272674738	24896.427267473777	24896.427267473777	0
C207_50	4	49055.51707580321	49055.5170758032	49055.5170758032	0

Instance	Routes	Published	0.11	0.12	ULP steps
C208_100	3	95551.43302101028	95551.43302101025	95551.43302101025	0
C208_25	2	24772.61520798644	24772.615207986444	24772.615207986444	0
C208_50	3	48799.019685982916	48799.019685982916	48799.019685982916	0
R101_100	20	26857.52618524431	26857.526185244315	26857.526185244315	0
R101_25	9	9079.419193454665	9079.419193454667	9079.419193454667	0
R101_50	13	16109.187250060135	16109.187250060135	16109.187250060135	0
R102_100	16	23356.164114669213	23356.164114669213	23356.164114669213	0
R102_25	7	7497.757904821816	7497.757904821817	7497.757904821817	0
R102_50	10	13450.010331939227	13450.01033193923	13450.01033193923	0
R103_100	14	20554.997580025138	20554.997580025127	20554.997580025127	0
R103_25	4	6551.8850636589605	6551.88506365896	6551.88506365896	0
R103_50	8	11689.331127988875	11689.331127988875	11689.331127988875	0
R104_100	12	18344.46945580322	18344.469455803224	18344.469455803224	0
R104_25	4	5935.653361538555	5935.653361538555	5935.653361538555	0
R104_50	5	10274.32585191496	10274.325851914957	10274.325851914957	0
R105_100	14	21802.178730919528	21802.17873091952	21802.178730919528	2
R105_25	5	7215.46184272281	7215.461842722811	7215.46184272281	1
R105_50	9	12805.527885400814	12805.527885400814	12805.527885400814	0
R106_100	13	20361.30992372795	20361.309923727957	20361.309923727957	0
R106_25	5	6537.447571293003	6537.4475712930025	6537.4475712930025	0
R106_50	8	11785.862370098512	11785.862370098512	11785.862370098512	0
R107_100	11	18890.956211034554	18890.956211034558	18890.956211034558	0
R107_25	4	5988.25993358735	5988.259933587349	5988.259933587349	0
R107_50	7	11022.483715561142	11022.483715561144	11022.483715561144	0
R108_100	9	17626.205886318035	17626.20588631804	17626.20588631804	0
R108_25	3	5676.841103437286	5676.841103437286	5676.841103437286	0
R108_50	5	10006.097370402033	10006.097370402034	10006.097370402034	0
R109_100	13	19235.566927507465	19235.56692750746	19235.56692750746	0
R109_25	6	6130.566921775302	6130.566921775303	6130.566921775303	0
R109_50	8	11302.708278653923	11302.708278653923	11302.708278653923	0
R110_100	12	18786.882466236002	18786.882466236006	18786.882466236002	1
R110_25	5	5934.458074488327	5934.458074488328	5934.458074488328	0
R110_50	7	10753.555581815852	10753.55558181585	10753.55558181585	0
R111_100	11	18386.624052201623	18386.62405220162	18386.62405220162	0
R111_25	4	6053.869456229827	6053.869456229827	6053.869456229828	1
R111_50	7	10805.473680753892	10805.473680753894	10805.473680753894	0
R112_100	11	17876.219351061358	17876.219351061354	17876.219351061354	0
R112_25	4	5627.463517108357	5627.463517108357	5627.463517108356	1
R112_50	6	10167.97223045511	10167.97223045511	10167.97223045511	0
R201_100	9	22389.299831918725	22389.299831918728	22389.299831918728	0
R201_25	5	7932.168490728331	7932.168490728329	7932.168490728331	2
R201_50	7	14051.502498920068	14051.502498920065	14051.502498920065	0
R202_25	5	7116.6875804462525	7116.687580446252	7116.687580446253	2
R202_50	4	12578.27964492557	12578.279644925566	12578.279644925566	0
R203_25	5	6694.055073106015	6694.055073106014	6694.055073106014	0
R203_50	6	11271.60636493839	11271.606364938389	11271.606364938389	0
R204_25	3	5792.277963212402	5792.2779632124	5792.277963212402	2
R205_100	7	19512.534398229916	19512.534398229913	19512.534398229913	0
R205_25	3	6435.533240013376	6435.533240013373	6435.533240013374	1
R205_50	4	11771.671308479528	11771.671308479521	11771.671308479523	1
R206_25	4	6121.820798795644	6121.82079879564	6121.820798795641	1
R206_50	4	10853.838560404427	10853.838560404402	10853.838560404398	2
R207_25	3	5701.448365232344	5701.448365232343	5701.448365232345	2
R208_25	3	5342.0830929657905	5342.083092965785	5342.083092965786	1
R209_25	3	5556.383372040877	5556.383372040877	5556.383372040877	0
R209_50	5	10111.837201669274	10111.837201669276	10111.837201669276	0
R210_25	3	6501.498569916836	6501.498569916836	6501.498569916834	2
R210_50	4	11091.08574690118	11091.085746901188	11091.085746901188	0
R211_25	2	4915.901249962704	4915.901249962704	4915.901249962702	2
R211_50	4	8910.200489621799	8910.200489621795	8910.200489621797	1
RC101_100	15	24778.29389604862	24778.29389604862	24778.29389604862	0
RC101_25	4	7038.613082210224	7038.613082210222	7038.613082210222	0
RC101_50	8	13996.907519207904	13996.907519207904	13996.907519207904	0
RC102_100	14	22689.966561961708	22689.966561961697	22689.966561961697	0
RC102_25	4	6187.160377361652	6187.160377361651	6187.160377361651	0
RC102_50	7	12439.517292766817	12439.517292766817	12439.517292766817	0
RC103_100	13	20680.0271367623	20680.027136762295	20680.027136762295	0
RC103_25	3	5447.369172519928	5447.369172519927	5447.369172519927	0
RC103_50	6	11329.650905119846	11329.650905119848	11329.650905119848	0
RC104_100	11	19476.349420257986	19476.349420257986	19476.349420257986	0
RC104_25	3	5199.950578854065	5199.950578854064	5199.950578854064	0
RC104_50	5	9842.805376068767	9842.805376068767	9842.805376068767	0
RC105_100	14	23376.81260716224	23376.812597162243	23376.812597162243	0

Instance	Routes	Published	0.11	0.12	ULP steps
RC105_25	4	6835.736855050216	6835.736855050216	6835.736855050216	0
RC105_50	8	13023.628336459238	13023.62833645924	13023.62833645924	0
RC106_100	14	21382.24298612069	21382.24298612069	21382.24298612069	0
RC106_25	3	5615.499154465348	5615.499154465348	5615.499154465347	1
RC106_50	6	11830.26377843958	11830.263778439577	11830.263778439577	0
RC107_100	13	20101.591901894477	20101.59190189448	20101.59190189448	0
RC107_25	3	5030.412031949936	5030.412031949936	5030.412031949936	0
RC107_50	6	10614.011179781179	10614.011179781175	10614.011179781175	0
RC108_100	13	19508.3746795146	19508.374679514607	19508.374679514607	0
RC108_25	3	4917.997007154809	4917.997007154808	4917.997007154808	0
RC108_50	6	10282.351509352056	10282.351509352055	10282.351509352055	0
RC201_100	7	24357.176039833932	24357.176039833925	24357.176039833925	0
RC201_25	4	8375.471791615142	8375.471791615146	8375.471791615142	2
RC201_50	6	14121.186503898034	14121.186503898034	14121.186503898036	1
RC202_100	8	22325.319356970922	22325.31935697092	22325.31935697092	0
RC202_25	1	7434.000076	7434.000076	7434.000076	0
RC202_50	3	13276.714854408838	13276.714854408858	13276.714854408858	0
RC203_25	2	6190.765421727738	6190.765421727737	6190.765421727736	1
RC204_25	2	5705.997863766978	5705.997863766982	5705.99786376698	2
RC205_100	10	23793.436950229338	23793.43695022933	23793.43695022933	0
RC205_25	3	7627.378483935336	7627.378483935336	7627.378483935336	0
RC205_50	4	13978.61796603659	13978.617966036589	13978.617966036589	0
RC206_25	2	6601.870995593264	6601.870995593265	6601.870995593265	0
RC206_50	3	12032.840668996923	12032.840668996927	12032.840668996929	1
RC207_25	3	5354.4998957631215	5354.499895763118	5354.49989576312	2
RC207_50	3	9726.395987575168	9726.395987575168	9726.395987575168	0
RC208_25	2	4352.797594907319	4352.797594907317	4352.797594907317	0
RC208_50	3	8362.996959514488	8362.996959514485	8362.996959514485	0

J Finite-input models and portability

This appendix provides the detailed input-model and reproducibility distinctions behind the main comparison.

J.1 The finite input model

A finite binary64 value is an exact dyadic rational, so an independent rational model can preserve the stored floating-point operands exactly. A decimal interpretation instead treats the shortest decimal representation as exact. For example, `Fraction(0.1)` and `Fraction("0.1")` denote different rationals. Neither reconstructs an unrounded Euclidean distance, and neither establishes the physical truth of a road travel-time estimate. Our TD references operate on materialized, frozen arrival-time breakpoints. The decimal TD reference uses their shortest round-trip decimal spellings, not an assertion that the original raw sidecar tokens or an ideal materializer denoted those exact decimals [Pyt26a].

We consequently separate 2 exact questions: the ideal piecewise-affine route defined by the binary operands, and the ideal piecewise-affine route defined by these decimal interpretations. Replacing the finite input model is different from changing the arithmetic on the same operands. This distinction also matters in static routing, where rounding a distance before summation and rounding its final sum define different objectives.

J.2 Conformance, fidelity and reproducibility

Operational conformance means reproducing the prescribed finite computation. *Mathematical fidelity* means proximity to a separately specified exact model, including its branch and feasibility decisions. *Reproducibility* means obtaining the same contract result from the same artifacts and recipe. A deterministic algorithm can have low fidelity at a discontinuity, and a more accurate algorithm can

break historical conformance. These properties should be measured separately.

Binary64 alone does not specify a recipe: parenthesization, intermediate rounding and fused operations matter. Our C++ diagnostic uses $a = 1 + 2^{-27}$, $b = 1 - 2^{-27}$ and $c = -1$. A separately rounded product followed by addition returns 0, while a fused multiply-add returns -2^{-54}. Both use floating-point hardware. A portable checker standard therefore fixes the operation sequence, rounding mode and contraction/reassociation policy, together with treatment of nonfinite and exceptional inputs. The reproduced examples use finite normal-range data [Gol91].

J.3 The role of a reference checker

A simple binary64 reference has practical value: solvers can port its operations, accepted moves can be repriced cheaply, and readers can audit one fixed recipe. Our prior KAYROS work deliberately places the checker above optimized move-evaluation structures and compares declared costs exactly. The manuscript and KAYROS reports describe sequential unnormalized composition and canonical TD summation as the standard. Their claims concern this prescribed computation and named solver assumptions. They do not imply unrestricted exact real arithmetic [Ras26b; Ras26a].

Changing that authority affects more than final cost strings: move acceptance, incumbent ranking, feasibility, lower bounds, reference gaps and anytime scores may all change. Prior comparisons with explicit reference policies remain evidence about the contract and reference snapshot that they used. A new checker value cannot be substituted into a historical result without revisiting that context [Ras26c; Ras26e].

K Provenance and reproduction

This appendix records the frozen sources and distribution checks, together with the evidence needed to reproduce the numerical comparisons. The public repository provides the executable code, compact TD input panels, historical static arc operands and archived route-level results. It retains upstream source and data licenses, while excluding personal discussions and the original private audit ledger [Ras26d].

The source commits are `425dff3` for 0.11.0 and `a9f7d9a` for 0.12.0. Annotated tag objects are recorded separately from their peeled source commits. The original corpus comparison uses MAMUT-routing baseline `93d9cc4` and snapshot `0d48b4f`, with 11 satellite baseline/snapshot pairs. These pins remain the authority for the original numerical results. Later static comparisons require identical instance bytes and report their candidate source revisions separately.

We download the published PyPI wheels into an isolated evidence cache, verify their advertised SHA-256 digests and compare their package Python files without installing them. All 23 files for 0.11.0 and all 25 for 0.12.0 match the corresponding frozen source exactly. Distribution URLs, hashes and upload timestamps appear in `manifests/distributions.json`. Hence the studied sources represent the published recipes, rather than an assumed tag-to-wheel correspondence.

The library and tools default branches contain no source changes beyond the frozen audited releases at the October 14 inspection. The newer main MAMUT-routing revision `6419553` changes its Mamut2026 submodule pin for a reported third BKS campaign. The pinned satellite revision `c62eca8` was unavailable from the satellite remote, whose advertised HEAD was `4715692`. We therefore exclude that campaign from numerical conclusions. Its absence does not alter results on the frozen corpus, but prevents describing that corpus as the latest published store. `manifests/recent-changes.json` records the inspection.

Historical source inspection dates the generic TD module to July 2, 2026. The fixed-cost duration objective was added in July and horizon/ready-time construction changed in August. Our reference

to the historical protocol concerns generic sequential composition, specifically the frozen 0.11 implementation, not byte-identical behavior of every earlier release.

The original investigation used a private 180-entry audit ledger as contextual evidence and reviewed 24 resolved items against pinned sources. We do not claim a new independent verification of all audit assertions. The public disposition map preserves the source paths, conclusions and review limits without publishing that ledger. Independent numerical measurements are distinguished from contextual audit claims.

The principal numerical evidence comprises `results/original-solutions.csv`, `results/original-routes.csv`, `results/static-solutions.csv`, `results/td-solutions.csv`, `results/td-replacements.csv`, `results/summary.json`, `results/static-history.json`, and the controlled timing JSON files. The archives in `data` restore the exact-panel inputs and all original route-result records. The bundle verifier checks their manifests and the frozen source hashes. Full-store materialized inputs require the pinned external repositories and their sidecars. Missing-input and source-sidecar manifests retain the coverage gaps.

From a clean checkout, run `uv sync -locked`, `uv run python -m ncci.bundle restore`, and `uv run pytest -q`. The repository README gives commands for candidate comparisons, conformance, controlled timings, table regeneration and the pdflatex/biber build. Timing reruns are host-specific. The report does not require private discussions or the audit ledger to reproduce its numerical comparisons.

L Historical static candidates and controlled measurements

This appendix records the additional comparison protocol and detailed measurements. Candidate history is read at the original frozen repository pins. A candidate is retained only if the instance bytes equal those of the corresponding frozen record and its routes pass the released structural and temporal checker. Repeated serialized route lists are counted once per instance. Historical declared values are not imposed on the new diagnostic score. The comparisons preserve each objective's fleet tier. Consumed arc operands, serialized routes, original declared costs, BKS byte hashes and source revisions are archived in `data/static-history.json.gz`.

The timing experiment runs serially, with no parallel experiment workers. It warms each operation, calibrates a repeated batch, disables garbage collection inside that batch and retains 7 round measurements. Release order alternates for TD and method order uses a fixed per-round shuffle for static reductions. A preloaded fold is first checked against its ordinary harness output. Complete checkers run in separate interpreters with model/ATF construction outside the measured call, and their feasibility, route durations, departures and solution totals are compared with frozen results. The benchmark records the interpreter, platform, clock and CPU affinity. These repeated measurements are descriptive on one host, not confidence intervals for a machine population.

Static selection takes the first, quarter, middle, three-quarter and last size-sorted record in each family, with path order breaking size ties. Reduction kernels use consumed arc vectors. Rational kernel inputs are converted outside the timed sum, while conversion-inclusive methods use the released exact decimal primitive or Python's binary Fraction conversion. The ranking comparison forms both objectives and preserves the fleet tier. Integer methods are eligible only when every scaled operand and the complete nonnegative total are integral and fit signed 64-bit range. Their timing uses Python arbitrary-size integers with conversion excluded, not a native signed-64-bit implementation. The 3 declared scales are 1, 1000 and 1000000.

Rational TD timing selects 12 routes per panel with at most 25 customers, using the minimum SHA-256 ordering of instance identifier, route index and the fixed seed string. Each input interpretation is evaluated 3 times with a 10-second per-call limit. Resource-limited cases remain explicit. Completed optima are checked against the archived exact reference before accepting the timing comparison.

Stage construction is included here, and the normal garbage-collection policy is retained for these direct calls. Unlike the calibrated reduction/checker batches, this bounded experiment includes each of its 3 calls without an additional warm-up. It is separate from the full exact-fidelity panel.

Table 8: Static timing panel. Medians in microseconds. Integer column uses scale 1000 where eligible. Decimal sum excludes operand conversion. Ranking ratio includes conversion and reduction of both candidates. A dash denotes ineligibility.

Family	Record	Arcs	Flat	`fsum`	Integer	Decimal	Rank ratio
Mamut2026	toronto/100	108	1.65	0.97	2.46	60.49	191.8
Mamut2026	arles/179	190	2.92	1.75	4.43	108.23	200.7
Mamut2026	pau/327	374	5.43	3.04	8.14	210.62	207.4
Mamut2026	rennes/599	714	10.79	6.18	15.48	408.77	212.6
Mamut2026	singapore/4000	4498	64.59	55.77	110.76	2807.36	239.5
Poryos2026	hong_kong/10	13	0.28	0.18	0.35	7.28	111.9
Poryos2026	hong_kong/25	29	0.51	0.30	0.71	16.60	148.2
Poryos2026	hong_kong/100	109	1.76	0.96	2.49	62.76	193.0
Poryos2026	hong_kong/500	543	8.11	4.58	11.86	305.68	212.6
Poryos2026	tokyo/1000	1079	16.52	9.98	24.11	619.36	212.7
Dimacs2021	C101	28	0.66	0.29	0.69	11.75	46.1
Dimacs2021	C106	110	2.53	1.00	2.55	46.37	52.5
Dimacs2021	C1_4_6	440	10.21	3.65	9.63	184.58	51.8
Dimacs2021	C1_8_3	873	20.40	7.39	19.50	374.10	53.5
Dimacs2021	RC2_10_9	1021	26.27	8.72	22.68	430.41	49.3
Ortec2022	cc05bba4/200	211	5.18	1.73	4.70	91.08	48.9
Ortec2022	bdfcbd22/247	258	6.59	1.98	5.87	112.11	48.3
Ortec2022	addee14a/302	318	8.89	2.73	7.57	155.04	49.6
Ortec2022	df8f2eda/359	379	10.87	3.19	9.24	182.87	49.0
Ortec2022	dd43a785/880	919	26.47	8.98	21.85	430.50	48.2
Sintef2008	C101	28	0.57	0.31	–	21.83	152.5
Sintef2008	C106	110	1.92	0.93	–	82.88	203.1
Sintef2008	C1_4_6	440	7.35	3.71	–	425.39	251.5
Sintef2008	C1_8_3	872	14.50	7.46	–	852.68	265.9
Sintef2008	RC2_10_9	1018	17.33	9.23	–	1031.18	264.0

The full identifiers, every method's raw batches and median timings are available in `results/controlled-static.json`. The timing column labels describe different finite computations, so a faster method is not automatically a conforming replacement. The native `fsum` routine and the Python loops also differ in implementation language.

For the original TD route panel, the paired 0.12/0.11 ratio has quartiles 0.644, 0.668 and 0.689. These describe route-to-route variation. Raw repeated samples and complete-checker setup fields remain in `results/controlled-td.json`.

For the broader TD route panel, the paired 0.12/0.11 ratio has quartiles 0.649, 0.698 and 0.719. These describe route-to-route variation. Raw repeated samples and complete-checker setup fields remain in `results/controlled-td.json`.

Table 9: All 31 historical pairs on which submitted-order scoring distinguishes a decimal tie. Gaps are current minus preceding candidate. Canonical gaps isolate the retained storage-order convention.

City	Metric	Customers	Submitted gap	Canonical gap
arles	euclidean	179	3.49e-10	5.82e-11
athens	euclidean	105	1.46e-10	2.91e-11
barcelona	euclidean	107	5.82e-11	0
berlin	euclidean	145	1.16e-10	-2.33e-10
caen	euclidean	192	4.66e-10	-1.16e-10
clermont_ferrand	euclidean	129	2.33e-10	1.75e-10
limoges	euclidean	152	2.91e-11	-2.91e-11
new_york	euclidean	142	8.15e-10	3.49e-10
paris	euclidean	171	2.91e-11	2.91e-11
perpignan	euclidean	132	5.82e-11	1.16e-10
reykjavik	euclidean	138	8.73e-11	5.82e-11
sydney	euclidean	118	2.33e-10	0
toronto	euclidean	100	3.49e-10	-1.16e-10
caen	fastest	192	2.91e-11	3.64e-12
manila	fastest	221	5.09e-11	0
montreal	fastest	112	7.28e-12	0
new_york	fastest	142	7.28e-12	0
reykjavik	fastest	138	1.46e-11	0
saint_malo	fastest	260	1.82e-11	7.28e-12
sydney	fastest	118	3.64e-11	0
toulon	fastest	196	2.91e-11	3.64e-12
athens	shortest	105	4.07e-10	0
berlin	shortest	145	2.33e-10	0
london	shortest	115	4.66e-10	0
new_york	shortest	142	2.33e-10	0
paris	shortest	171	1.46e-10	0
perpignan	shortest	132	2.91e-10	5.82e-11
reykjavik	shortest	138	1.16e-10	0
rome	shortest	102	3.49e-10	0
sydney	shortest	118	5.82e-10	0
toronto	shortest	100	4.66e-10	0